

Project Structure and Artefacts

Resilio — Mobile Mental Health Platform for Nepal | Milan Shrestha | ID: 23057135 |
03_Construction/Project_Structure_and_Artefacts.md

Project: Resilio — Mobile Mental Health Platform for Nepal

Student: Milan Shrestha | **ID:** 23057135

Methodology: Rational Unified Process (RUP) — 4 phases, 20 weeks

This document provides a complete map of the project structure — the Flutter codebase, the Node.js backend, the SQL migration files, the Protobuf definitions, and the Project Artefact folder hierarchy — all organised according to the RUP phases in which they were produced.

PART 1: FLUTTER MOBILE APPLICATION STRUCTURE

The Flutter project follows **Feature-first Clean Architecture** (Robert Martin, 2017). Each feature is a self-contained vertical slice with its own `data/`, `domain/`, and `presentation/` layers.

lib/	
■■■ main.dart	← App entry point; DI initialisation
■	
■■■ app/	← App-level bootstrap
■ ■■■ app.dart	← MaterialApp + GoRouter mount
■	
■■■ common/	← Shared UI primitives (all features)
■ ■■■ utils/	← Date formatters, validators, helpers
■ ■■■ widgets/	← Reusable UI components (buttons, cards)
■	
■■■ core/	← Cross-cutting infrastructure
■ ■■■ config/	← API endpoint constants, environment config
■ ■■■ constants/	← App-wide constants (colors, strings)
■ ■■■ database/	← SQLite offline database (sqflite)
■ ■■■ di/	← GetIt + Injectable DI container
■ ■ ■■■ injection.config.dart	← Auto-generated DI registrations
■ ■■■ errors/	← Failure types, Either wrappers (dartz)
■ ■■■ localization/	← ARB files; Nepali l10n
■ ■■■ network/	← Dio HTTP client + interceptors
■ ■■■ proto_generated/	← Dart classes generated from .proto files
■ ■■■ protos/	← .proto source definitions
■ ■■■ routing/	← GoRouter config + route guards
■ ■ ■■■ app_router.dart	← Route definitions; role-based redirects
■ ■ ■■■ route_names.dart	← Named route constants
■ ■■■ services/	← Cloudinary, FCM token, connectivity
■ ■ ■■■ cloudinary_service.dart	← Upload image/audio/video to Cloudinary
■ ■■■ settings/	← Shared preferences wrapper
■ ■■■ theme/	← AppTheme, color schemes, text styles
■ ■ ■■■ cubit/	← ThemeCubit for dark/light mode toggle
■ ■■■ usecases/	← Base UseCase abstract class
■ ■■■ widgets/	← Global widgets (loading indicators, dialogs)
■	
■■■ l10n/	← Localisation ARB files
■	
■■■ features/	← Feature modules (vertical slices)
■	
■■■ customer/	← CUSTOMER ROLE (all customer features)
■ ■■■ auth/	← Authentication (OTP, Google, Facebook)
■ ■ ■■■ data/	
■ ■ ■■■ ■■■ datasources/	
■ ■ ■■■ ■■■ ■■■ local/	← Token storage (shared_preferences)
■ ■ ■■■ ■■■ ■■■ remote/	← SuperTokens API calls
■ ■ ■■■ ■■■ ■■■ models/	← UserModel (json_serializable)
■ ■ ■■■ ■■■ ■■■ repositories/	← AuthRepositoryImpl
■ ■ ■■■ ■■■ domain/	
■ ■ ■■■ ■■■ ■■■ entities/	← User entity
■ ■ ■■■ ■■■ ■■■ repositories/	← AuthRepository interface
■ ■ ■■■ ■■■ ■■■ usecases/	← LoginWithOtp, LoginWithGoogle, Logout
■ ■ ■■■ ■■■ presentation/	
■ ■ ■■■ ■■■ ■■■ bloc/	← AuthCubit + AuthState (Freezed)
■ ■ ■■■ ■■■ ■■■ screens/	← SplashScreen, OtpScreen, OnboardingScreen
■ ■ ■■■ ■■■ ■■■ widgets/	← OTP input widget, social login buttons
■ ■	
■ ■■■ dashboard/	← Customer home dashboard

■ ■ ■■ presentation/	
■ ■ ■■ screens/	← CustomerDashboardScreen (tabs)
■ ■	
■ ■■ appointments/	← Customer appointment management
■ ■ ■■ presentation/	
■ ■ ■■ screens/	← AppointmentsListScreen, AppointmentDetailScreen
■ ■	
■ ■■ booking/	← Therapist booking flow
■ ■ ■■ presentation/	
■ ■ ■■ screens/	← SelectSlotScreen, BookingConfirmScreen
■ ■	
■ ■■ therapist/	← Therapist directory (customer view)
■ ■ ■■ presentation/	
■ ■ ■■ screens/	← TherapistListScreen, TherapistProfileScreen
■ ■	
■ ■■ audio/	← Audio guided session player
■ ■ ■■ data/	
■ ■ ■■ ■■ datasources/remote/	
■ ■ ■■ ■■ repositories/	
■ ■ ■■ domain/	
■ ■ ■■ ■■ entities/	← AudioTrack entity
■ ■ ■■ ■■ repositories/	
■ ■ ■■ presentation/	
■ ■ ■■ bloc/	
■ ■ ■■ screens/	← AudioPlayerScreen (just_audio)
■ ■	
■ ■■ video/	← Video content player
■ ■ ■■ data/	
■ ■ ■■ ■■ datasources/remote/	
■ ■ ■■ ■■ repositories/	
■ ■ ■■ domain/	
■ ■ ■■ ■■ entities/	← VideoTrack entity
■ ■ ■■ ■■ repositories/	
■ ■ ■■ presentation/	
■ ■ ■■ bloc/	
■ ■ ■■ ■■ long_video/	← LongVideoCubit
■ ■ ■■ ■■ short_video/	← ShortVideoCubit (Reels)
■ ■ ■■ screens/	← VideoPlayerScreen (better_player_plus)
■ ■ ■■ widgets/	
■ ■	
■ ■■ categories/	← Content category browsing
■ ■ ■■ data/	
■ ■ ■■ ■■ datasources/remote/	
■ ■ ■■ ■■ models/	
■ ■ ■■ ■■ repositories/	
■ ■ ■■ domain/	
■ ■ ■■ ■■ entities/	
■ ■ ■■ ■■ repositories/	
■ ■ ■■ ■■ usecases/	
■ ■ ■■ presentation/	
■ ■ ■■ bloc/	
■ ■ ■■ screens/	← ContentHubScreen, ArticleReaderScreen
■ ■	
■ ■■ tips/	← Wellness tips

```

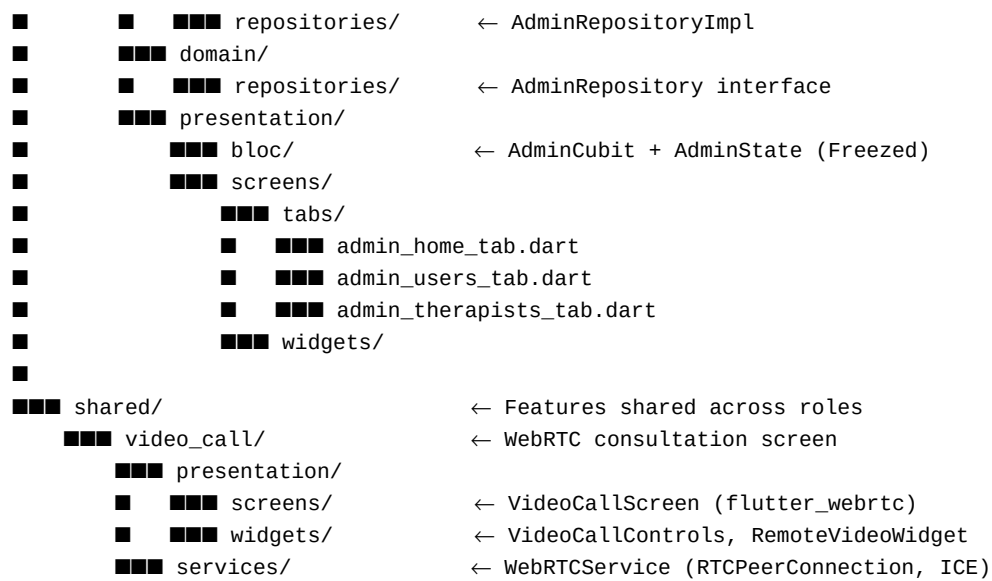
■ ■ ■ data/
■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ repositories/
■ ■ ■ domain/
■ ■ ■ ■ entities/
■ ■ ■ ■ repositories/
■ ■ ■ presentation/
■ ■ ■ ■ bloc/
■ ■ ■ ■ screens/
■ ■ ■ ■ widgets/
■ ■
■ ■ ■ profile/ ← Customer profile and achievements
■ ■ ■ presentation/
■ ■ ■ ■ bloc/
■ ■ ■ ■ screens/ ← ProfileScreen, AchievementGalleryScreen
■ ■ ■ ■ widgets/
■ ■
■ ■ ■ subscription/ ← Subscription plans and eSewa payment
■ ■ ■ data/
■ ■ ■ ■ datasources/
■ ■ ■ ■ models/
■ ■ ■ ■ repositories/
■ ■ ■ domain/
■ ■ ■ ■ entities/
■ ■ ■ ■ repositories/
■ ■ ■ presentation/
■ ■ ■ ■ bloc/
■ ■ ■ ■ screens/ ← SubscriptionPlansScreen, PaymentScreen
■ ■
■ ■ ■ settings/ ← User settings and preferences
■ ■ ■ data/
■ ■ ■ ■ datasources/
■ ■ ■ ■ repositories/
■ ■ ■ domain/
■ ■ ■ ■ entities/
■ ■ ■ ■ repositories/
■ ■ ■ presentation/
■ ■ ■ ■ bloc/
■ ■ ■ ■ screens/ ← SettingsScreen, NotificationPrefsScreen
■ ■ ■ ■ widgets/
■ ■
■ ■ ■ splash/ ← Splash screen (auth state check)
■ ■ ■ presentation/
■ ■ ■ ■ screens/ ← SplashScreen
■
■ ■ ■ therapist/ ← THERAPIST ROLE (all therapist features)
■ ■ ■ dashboard/ ← Therapist dashboard (tabs + cubit)
■ ■ ■ data/
■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ repositories/
■ ■ ■ domain/
■ ■ ■ ■ entities/
■ ■ ■ ■ repositories/
■ ■ ■ presentation/

```

```

■ ■ ■ ■ bloc/                ← TherapistCubit + TherapistState (Freezed)
■ ■ ■ ■ screens/
■ ■ ■ ■ ■ tabs/
■ ■ ■ ■ ■ content/          ← AddEditContentSheet
■ ■ ■ ■ ■ therapist_home_tab.dart
■ ■ ■ ■ ■ therapist_appointments_tab.dart
■ ■ ■ ■ ■ therapist_patients_tab.dart
■ ■ ■ ■ ■ therapist_content_hub_tab.dart
■ ■ ■ ■ ■ therapist_earnings_tab.dart
■ ■ ■ ■ widgets/
■ ■ ■ ■ ■ appointments/      ← Therapist appointment management
■ ■ ■ ■ ■ data/
■ ■ ■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ ■ domain/
■ ■ ■ ■ ■ ■ entities/
■ ■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ ■ (presentation in dashboard/screens/tabs/)
■ ■ ■ ■ content/            ← Content publishing
■ ■ ■ ■ data/
■ ■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ domain/
■ ■ ■ ■ ■ entities/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ (presentation in dashboard/screens/tabs/content/)
■ ■ ■ ■ earnings/          ← Earnings summary
■ ■ ■ ■ data/
■ ■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ domain/
■ ■ ■ ■ ■ entities/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ (presentation in dashboard/screens/tabs/)
■ ■ ■ ■ patients/          ← Patient list
■ ■ ■ ■ data/
■ ■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ domain/
■ ■ ■ ■ ■ entities/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ (presentation in dashboard/screens/tabs/)
■ ■ ■ ■ profile/           ← Therapist profile management
■ ■ ■ ■ data/
■ ■ ■ ■ ■ datasources/remote/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ domain/
■ ■ ■ ■ ■ entities/
■ ■ ■ ■ ■ repositories/
■ ■ ■ ■ (presentation in dashboard/screens/tabs/)
■
■ ■ ■ ■ admin/              ← ADMIN ROLE (all admin features)
■ ■ ■ ■ ■ dashboard/        ← Admin dashboard (tabs + cubit)
■ ■ ■ ■ ■ data/

```



Clean Architecture Layer Responsibilities

Layer	Responsibility	Dependencies Allowed
`domain/entities/`	Pure Dart business objects — no Flutter imports	None
`domain/repositories/`	Abstract interfaces (contracts)	entities only
`domain/usecases/`	Business logic — calls repository interfaces	repositories, entities
`data/repositories/`	Implements domain repository; orchestrates data sources	datasources, models
`data/datasources/`	Raw HTTP calls (Dio) or local DB (sqlite)	Dio, sqlite
`data/models/`	JSON-deserialisable versions of entities (json_serializable)	entities
`presentation/bloc/`	Cubit state management; calls use cases	usecases, Freezed states
`presentation/screens/`	Flutter Widgets; renders state	BLoC/Cubit, Widgets

Key Core Files

File	Purpose
`core/di/injection.config.dart`	Auto-generated by build_runner; registers all GetIt bindings
`core/routing/app_router.dart`	GoRouter config; role guards redirect unauth users to login
`core/routing/route_names.dart`	String constants for all named routes
`core/network/dio_client.dart`	Dio HTTP client; attaches SuperTokens Bearer token via interceptor
`core/services/cloudinary_service.dart`	Upload and retrieve media from Cloudinary CDN
`core/database/`	SQLite offline database for mood entries (sqflite)
`core/protos/`	`.proto` source files (offline mood entry serialisation)
`core/proto_generated/`	Dart classes generated from `proto` files via protoc_plugin

PART 2: NODE.JS BACKEND STRUCTURE

The backend follows **Clean Architecture** with Express as the HTTP transport layer. Routes → Controllers → Domain Services → Data Repositories.

```

Backend/
  ■■■ src/
  ■   ■■■ index.js           ← Express app bootstrap; middleware mount; route
    registration
  ■   ■
  ■   ■■■ config/
  ■   ■   ■■■ supabase-client.js   ← Supabase JS client initialisation
  ■   ■   ■■■ supertokens.js      ← SuperTokens backend SDK configuration
  ■   ■
  ■   ■■■ middlewares/
  ■   ■   ■■■ auth-middleware.js   ← SuperTokens verifySession() + requireRole()
  ■   ■   ■■■ proto-middleware.js ← Protobuf request/response handling
  ■   ■
  ■   ■■■ routes/                ← Express Routers (URL prefix → controller)
  ■   ■   ■■■ passwordless-routes.js ← /auth/passwordless/*
  ■   ■   ■■■ user-routes.js      ← /users/*
  ■   ■   ■■■ therapist-routes.js ← /therapists/* (public)
  ■   ■   ■■■ therapist-portal-routes.js ← /therapist-portal/* (therapist auth required)
  ■   ■   ■■■ appointment-routes.js ← /appointments/*
  ■   ■   ■■■ payment-routes.js    ← /payment/*
  ■   ■   ■■■ subscription-routes.js ← /subscriptions/*
  ■   ■   ■■■ audio-routes.js      ← /audio/*
  ■   ■   ■■■ video-routes.js      ← /videos/*
  ■   ■   ■■■ images-routes.js     ← /images/*
  ■   ■   ■■■ category-routes.js   ← /categories/*
  ■   ■   ■■■ quote-routes.js      ← /quotes/*
  ■   ■   ■■■ tips-routes.js       ← /tips/*
  ■   ■   ■■■ games-routes.js      ← /games/*
  ■   ■   ■■■ favorite-routes.js   ← /favorites/*
  ■   ■   ■■■ notification-routes.js ← /notifications/*
  ■   ■   ■■■ preference-routes.js ← /preferences/*
  ■   ■   ■■■ webrtc-signal-routes.js ← /signal/* (WebRTC room management REST)
  ■   ■   ■■■ admin-routes.js      ← /admin/* (admin auth required)
  ■   ■
  ■   ■■■ controllers/           ← Request parsing → use case call → response
  ■   ■   ■■■ passwordless-controller.js
  ■   ■   ■■■ user-controller.js
  ■   ■   ■■■ therapist-controller.js
  ■   ■   ■■■ therapist-portal-controller.js
  ■   ■   ■■■ appointment-controller.js
  ■   ■   ■■■ appointment-messages-controller.js
  ■   ■   ■■■ payment-controller.js
  ■   ■   ■■■ subscription-controller.js
  ■   ■   ■■■ audio-controller.js
  ■   ■   ■■■ video-controller.js
  ■   ■   ■■■ images-controller.js
  ■   ■   ■■■ category-controller.js
  ■   ■   ■■■ quote-controller.js
  ■   ■   ■■■ tips-controller.js
  ■   ■   ■■■ games-controller.js
  ■   ■   ■■■ favorite-controller.js
  ■   ■   ■■■ notification-controller.js
  ■   ■   ■■■ preference-controller.js
  ■   ■   ■■■ webrtc-signal-controller.js

```

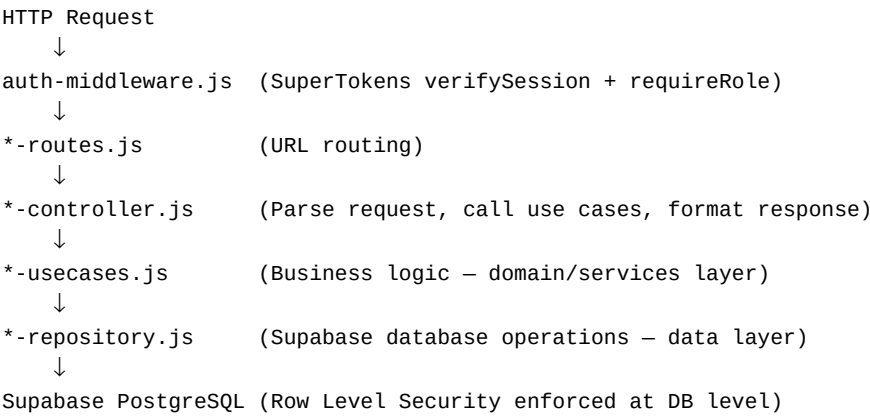
```

■ ■ ■■■ admin-controller.js
■ ■
■ ■■■ domain/
■ ■ ■■■ services/ ← Business logic use cases (pure functions)
■ ■ ■■■ user-usecases.js
■ ■ ■■■ audio-usecases.js
■ ■ ■■■ video-usecases.js
■ ■ ■■■ images-usecases.js
■ ■ ■■■ category-usecases.js
■ ■ ■■■ quote-usecases.js
■ ■ ■■■ tips-usecases.js
■ ■ ■■■ favorite-usecases.js
■ ■ ■■■ preference-usecases.js
■ ■
■ ■■■ data/
■ ■ ■■■ repositories/ ← Supabase database calls
■ ■ ■■■ user-repository.js
■ ■ ■■■ user-preference-repository.js
■ ■ ■■■ therapist-repository.js
■ ■ ■■■ audio-repository.js
■ ■ ■■■ video-repository.js
■ ■ ■■■ images-repository.js
■ ■ ■■■ category-repository.js
■ ■ ■■■ quote-repository.js
■ ■ ■■■ tips-repository.js
■ ■ ■■■ favorite-repository.js
■ ■ ■■■ preference-repository.js
■ ■
■ ■■■ services/ ← External service integrations
■ ■ ■■■ fcm-service.js ← firebase-admin SDK – send FCM push notifications
■ ■ ■■■ notification-service.js ← Notification creation + FCM trigger orchestrator
■ ■ ■■■ webrtc-socket.js ← Socket.IO event handlers for WebRTC signalling
■
■■■ sql/ ← PostgreSQL migration files (run in order)
■ ■■■ 01_users_table.sql
■ ■■■ 02_preferences_tables.sql
■ ■■■ 03_categories_table.sql
■ ■■■ 04_audio_tracks_table.sql
■ ■■■ 04_quotes_table.sql
■ ■■■ 05_tips_table.sql
■ ■■■ 05_video_tracks_functions.sql
■ ■■■ 05_video_tracks_table.sql
■ ■■■ 06_images_table.sql
■ ■■■ 07_favorites_table.sql
■ ■■■ 08_games_tables.sql
■ ■■■ 09_subscriptions_table.sql
■ ■■■ 10_appointments_table.sql
■ ■■■ 11_therapist_profiles_table.sql
■ ■■■ 12_supertokens_migration.sql
■ ■■■ 13_notifications_table.sql
■ ■■■ 14_appointments_price_and_fixes.sql
■ ■■■ 15_appointment_messages_table.sql
■ ■■■ 16_users_profile_fields.sql
■ ■■■ 17_video_comments_table.sql

```

```
■ ■■■ 18_games_achievements_seed.sql
■ ■■■ 19_quiz_affirmation_tables.sql
■
■■■ protos/                                ← Protobuf definitions (shared with Flutter)
■ ■■■ common.proto
■ ■■■ auth.proto
■ ■■■ user.proto
■ ■■■ therapist.proto
■ ■■■ appointment.proto
■ ■■■ audio.proto
■ ■■■ video.proto
■ ■■■ images.proto
■ ■■■ category.proto
■ ■■■ quote.proto
■ ■■■ tips.proto
■ ■■■ subscription.proto
■ ■■■ games.proto
■
■■■ vercel.json                            ← Vercel deployment config (routes /api/(.*) →
    src/index.js)
```

Backend Architecture Flow



PART 3: DATABASE MIGRATION SEQUENCE

Each SQL file in Backend/sql/ corresponds to a specific database feature. They must be executed in order.

Migration	File	Table / Change	RUP Phase
01	`01_users_table.sql`	`users` — base user records	Elaboration

Migration	File	Table / Change	RUP Phase
02	`02_preferences_tables.sql`	`user_preferences`	Elaboration
03	`03_categories_table.sql`	`categories`	Construction Iter 2
04a	`04_audio_tracks_table.sql`	`audio_tracks`	Construction Iter 2
04b	`04_quotes_table.sql`	`quotes`	Construction Iter 2
05a	`05_tips_table.sql`	`tips`	Construction Iter 2
05b	`05_video_tracks_table.sql`	`video_tracks`	Construction Iter 2
05c	`05_video_tracks_functions.sql`	DB functions for video	Construction Iter 2
06	`06_images_table.sql`	`images`	Construction Iter 2
07	`07_favorites_table.sql`	`favorites`	Construction Iter 5
08	`08_games_tables.sql`	`game_sessions`, `achievements`	Construction Iter 7
09	`09_subscriptions_table.sql`	`subscriptions`	Construction Iter 7
10	`10_appointments_table.sql`	`appointments`	Construction Iter 4
11	`11_therapist_profiles_table.sql`	`therapist_profiles`	Construction Iter 3
12	`12_supertokens_migration.sql`	SuperTokens schema sync	Construction Iter 1
13	`13_notifications_table.sql`	`notifications`	Construction Iter 7
14	`14_appointments_price_and_fixes.sql`	Alter `appointments` — add price	Construction Iter 4
15	`15_appointment_messages_table.sql`	`appointment_messages`	Construction Iter 6
16	`16_users_profile_fields.sql`	Alter `users` — add profile fields	Construction Iter 3
17	`17_video_comments_table.sql`	`video_comments`	Construction Iter 6

Migration	File	Table / Change	RUP Phase
18	`18_games_achievement s_seed.sql`	Seed achievement records	Construction Iter 7
19	`19_quiz_affirmation_tabl es.sql`	`quiz_questions`, `affirmations`	Construction Iter 7

PART 4: PROTOBUF DEFINITIONS

Protobuf .proto files live in both `Backend/protos/` (Node.js side) and `lib/core/protos/` (Flutter side). The Flutter Dart classes are generated into `lib/core/proto_generated/` via `protoc_plugin`.

Proto File	Purpose
`common.proto`	Shared message types (timestamps, pagination)
`auth.proto`	Auth request/response messages
`user.proto`	User profile message; offline user data
`therapist.proto`	Therapist profile message
`appointment.proto`	Appointment request/response; offline sync
`audio.proto`	Audio track message
`video.proto`	Video track message
`images.proto`	Image record message
`category.proto`	Content category message
`quote.proto`	Daily quote message
`tips.proto`	Wellness tip message
`subscription.proto`	Subscription plan and status message
`games.proto`	Game session and achievement message

The primary use of Protobuf in Resilio is **offline mood entry serialisation**: when the device is offline, mood log entries are serialised into binary Protobuf format and stored in SQLite (`sqlite`). On reconnection, the sync service deserialises and uploads them to Supabase.

PART 5: PROJECT ARTEFACT FOLDER — RUP PHASE MAPPING

The Project Artefact/ folder mirrors the four RUP phases. Each folder contains a README.md and week-by-week sub-folders documenting what was produced.

FYP Report Writing/	
■■■ Project Artefact/	
■■■ README.md	← Master directory with full folder tree
■	
■■■ 01 Feasibility Study/	← RUP INCEPTION (Weeks 1-5)
■ ■■■■ README.md	← Phase overview, key decisions
■ ■■■■ Gantt Chart Drafts/	← Initial and Final Gantt charts
■ ■■■■ Week - 01/README.md	← Project setup, problem framing
■ ■■■■ Week - 02/README.md	← Stakeholder interviews, pre-survey launch
■ ■■■■ Week - 03/README.md	← Pre-survey results, feasibility report
■ ■■■■ Week - 04/README.md	← Risk register, WBS, environment setup
■ ■■■■ Week - 05/README.md	← Technology selection, Inception wrap-up
■	
■■■ 02 Requirement Analysis/	← RUP ELABORATION (Weeks 6-10)
■ ■■■■ README.md	← Phase overview, artefacts
■ ■■■■ Week - 06/README.md	← SRS, use case diagrams, ERD draft
■ ■■■■ Week - 07/README.md	← Architecture diagram, wireframes, DFDs
■ ■■■■ Week - 08/README.md	← SuperTokens + eSewa prototypes
■ ■■■■ Week - 09/README.md	← WebRTC prototype, Figma mockups
■ ■■■■ Week - 10/README.md	← Interim report, RLS policies
■	
■■■ 03 Design/	← RUP CONSTRUCTION Iterations 1-4 (Weeks 11-14)
■ ■■■■ README.md	← Phase overview, iteration goals
■ ■■■■ Week - 11/README.md	← Auth module, GoRouter, onboarding flows
■ ■■■■ Week - 12/README.md	← DB schema live, API structure, RLS policies
■ ■■■■ Week - 13/README.md	← Therapist directory, admin verification
■ ■■■■ Week - 14/README.md	← Appointment booking, eSewa integration
■	
■■■ 04 Development/	← RUP CONSTRUCTION Iterations 5-7 (Weeks 15-19)
■ ■■■■ README.md	← Phase overview, full module tree
■ ■■■■ Week - 15/README.md	← WebRTC consultation, Socket.IO signalling
■ ■■■■ Week - 16/README.md	← Mood tracking, PHQ-9/GAD-7, Clouldinary
■ ■■■■ Week - 17/README.md	← Content hub, subscription gating
■ ■■■■ Week - 18/README.md	← Wellness games, achievement system
■ ■■■■ Week - 19/README.md	← Offline sync, FCM, subscriptions
■	
■■■ 05 Testing and Implementation/	← RUP TRANSITION (Week 20)
■■■ README.md	← Phase overview, deployment summary
■■■ Week - 20/	
■■■ README.md	← System tests, UAT, post-survey, deployment
■■■ Research/	← Post-survey raw data (132 responses)
■■■ Customer Validation/	← UAT feedback forms (7 participants)
■■■ Social, Ethical and Legal Issues/	

What Each Week README Contains

Every week-level README follows a consistent structure:

Features Built — table of features, layer, and status

Code Snippet — representative code from the iteration (not pseudo-code — actual project code)

Test Cases Passed — list of TC-U / TC-S / TC-UAT IDs validated in that week

Files to Place Here — checklist for screenshots and evidence to add physically to the folder

PART 6: WEEK-BY-WEEK FEATURE DELIVERY (RUP ITERATIONS)

Week	RUP Phase	Iteration	Primary Deliverable
1	Inception	—	Project proposal, problem statement, mind map
2	Inception	—	Stakeholder interviews, pre-survey design
3	Inception	—	Pre-survey results (166 responses), feasibility study
4	Inception	—	Risk register, WBS, development environment setup
5	Inception	—	Technology selection (Flutter, Node.js, Supabase, SuperTokens, eSewa)
6	Elaboration	—	SRS, use case diagrams (3×), ERD v1
7	Elaboration	—	Architecture diagram, wireframes (7 screens), DFDs
8	Elaboration	—	SuperTokens OTP prototype, eSewa payment prototype
9	Elaboration	—	WebRTC peer connection prototype, Figma UI mockups

Week	RUP Phase	Iteration	Primary Deliverable
10	Elaboration	—	Interim Report, RLS policies, Elaboration review
11	Construction	Iter 1	SuperTokens OTP auth, Google/Facebook Sign-In, GoRouter, onboarding
12	Construction	Iter 2	Supabase schema deployed, RLS policies live, API skeleton
13	Construction	Iter 3	Therapist directory, admin verification workflow, profile screens
14	Construction	Iter 4	Appointment booking, real-time slot availability, eSewa payment
15	Construction	Iter 5	WebRTC consultation (flutter_webrtc), Socket.IO signalling, admin dashboard
16	Construction	Iter 6	Mood tracking, PHQ-9/GAD-7, crisis alert, Cloudinary integration
17	Construction	Iter 6 cont.	Content hub (articles/audio/video), subscription gating, content moderation
18	Construction	Iter 7	Wellness games (4 types), achievement system, favourites, text-to-speech
19	Construction	Iter 7 cont.	SQLite offline sync, FCM push notifications, subscription plans
20	Transition	—	System tests (40), UAT (25), post-survey (132), Vercel deployment, final report

PART 7: CROSS-REFERENCE — CODE TO ARTEFACT TO REPORT

Code Location	Artefact Folder	Report Section
`lib/features/customer/auth/`	`03 Design/Week - 11/`	Chapter 3, Section 3.5.1
`lib/features/shared/video_call/`	`04 Development/Week - 15/`	Chapter 3, Section 3.5.5
`lib/features/customer/settings/` + subscription	`04 Development/Week - 19/`	Chapter 3, Section 3.5.8
`Backend/src/routes/appointment-r outes.js`	`03 Design/Week - 14/`	Chapter 3, Section 3.5.4
`Backend/src/services/webrtc-sock et.js`	`04 Development/Week - 15/`	Chapter 3, Section 3.5.5; Appendix J
`Backend/sql/` (all 19 migrations)	`03 Design/Week - 12/`	Chapter 3, Section 3.4; Appendix M
`lib/core/protos/` + `Backend/protos/`	`04 Development/Week - 19/`	Chapter 3, Section 3.5.9
`05 Testing and Implementation/Week - 20/`	System + UAT results	Chapter 4; Appendix K